

Small models. Tight loops.

What a coding-agent harness is actually doing for you

one Python file

real compiler

external postconditions

Denis Akhiyarov · Aug 1, 2026

```
nanagent / run 01
```

```
$ askme "build a two-file C program"
```

```
plan   write .h → write .c → compile → run
```

```
tool   cc -o main main.c
```

```
check  ./main
```

AGENT_OK

The bug that ate minutes

```
// missing: #include <stdio.h>
int main() {
    printf("FIXED\n");
    return 0;
}

error: implicit declaration of printf
```

step 1

Compile

Useful failure: the compiler names the bug.

next

Bad edit

Malformed JSON or the wrong exact-match string.

140–253s

Think, retry, re-read

The model spends tokens rediscovering known state.

Wrong move: buy more thinking. **Better move:** turn the compiler error into control flow.

Move reliability out of the model



```
compiler stderr → [compile_error] → repair / retry → ./main == "AGENT_OK"
```

Four models, two concrete jobs

MULTI-FILE BUILD

Header + source → compile → run

```
msg.h + main.c · ./main == REPLAN_OK
```

REPAIR

Fix a Python syntax error

```
python3 greet.py exits 0 · stdout contains hello
```

Gemma 4 26B A4B

MoE · 3.8B active

Gemma 4 31B

dense · 30.7B

Qwen3.6-27B

dense · 27B

Qwen3.6-35B-A3B

MoE · 3B active

Held fixed: NanAgent implementation, task prompt, SiliconFlow-only routing, one retry policy, strict external checks. Unique catalog matches were FP8. **n=1/cell**. 31B was a post-hoc extension.

What happened?

Model	2-file build	Repair	Responses	Billed credits
Gemma 4 26B A4B	PASS 603.6s	PASS 20.0s	39	\$0.004147
Gemma 4 31B	PASS 66.5s	PASS 22.4s	19	\$0.001321
Qwen3.6-27B	PASS 47.9s	PASS 23.0s	22	\$0.004968
Qwen3.6-35B-A3B	FAIL 17.7s	PASS 11.8s	14	\$0.001875

Pass requires pytest, **agent complete**, and the external check. “Responses” means usage-bearing model replies; raw HTTP attempts were not instrumented.

7/8 passed. Gemma: both 2/2 · 31B build used 8 vs 29 responses · repair used 11 vs 10. Size + architecture changed.

Useful evidence, small claim

What we actually verify

- Structured actions survive the model/provider path
- The loop can build or repair these exact programs
- The resulting artifact executes correctly
- Responses, tokens, route, and cost are auditable

What we do not claim

- Full-app or long-horizon reliability
- Local-laptop speed from hosted runs
- UX, architecture, or maintainability quality
- Model size alone caused the Gemma gap
- Contest-code scores predict agent performance

TAKEAWAY

The loop is the product

Model

Typed action

Guardrail

Real tool

Postcondition

**Small models do not need easier standards.
They need tighter feedback loops.**

github.com/den-run-ai/askme · slides, blog, protocol, and raw summary data